

P1274R0

Bang For The Buck

Published Proposal, 2018-07-15

This version:

<https://wg21.link/p1274r0>

Author:

[Isabella Muerte](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1274R0](#)

Current Source:

slurps-mad-rips/papers/proposals/bang-for-the-buck.bs

Abstract

We should give C++ programmers the ability to use additional characters in identifiers

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 Wording

4.1 ! and ?

4.2 \$

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

Despite the vast number of characters now allotted to the C++ standard regarding identifiers, the one character that is continually seen in extensions is the ASCII character `$`. While some want to permit this as an operator `reflexpr`, it is the author's opinion that it makes more sense to permit it as an identifier in functions, namespaces, classes, and variables.

While there are some concerns regarding permitting its use in identifiers, this paper does layout a solution for vendors who have supported this extension on some platforms up until now, while also laying a foundation for future characters that exist on all keyboards but might cause linker issues with older platforms.

Additionally, this paper seeks to permit adding both the `!` and `?` tokens at the end of member functions. This would permit calls such as `ptr.reset!()`, and `vector.empty?()`, which could be used to reduce confusion when a function might be a modifier vs an observer.

§ 3. Design

While several vendors have permitted the use of the `$` in the past, it is not able to be supported on all platforms due to linker requirements. While the C++ standard does not have a true notion of "a linker", there is still the reality that at the end of the day we need to combine our translation units into *something*. Because of this, this paper takes a unique route for representing the `$` in sources. Effectively, we do not add `$` to the basic source character set. Instead, we permit the preprocessor during the 1st phase of translation to turn the `$` into its *universal-character-name*, thus rendering it into the value `\u0024`. Current implementations are then free to mangle the resulting identifier as

though it were a unicode character. For platforms that have supported \$ as an extension, they are free to generate symbols for both the unicode and \$ literal character.

Both ! and ? are part of the *basic-execution-set* and therefore are being repurposed for this specific identifier location.

§ 4. Wording

All wording is relative to [\[N4762\]](#).

Note: Wording for the exact changes to permit ! and ? are currently withheld until the San Diego post mailing to see where they should be placed exactly within the grammar.

§ 4.1. ! and ?

Insert into 5.10 Identifiers [**lex.name**]

identifier

identifier-nondigit

identifier identifier-nondigit

identifier digit

identifier-special:

identifier identifier-special-char

identifier-special-char: one of

! ?

§ 4.2. \$

Insert into Table 2 in 5.10 Identifiers [**lex.name**]

<u>0024</u>				
00A8	00AA	00AD	00AF	00B2-00B5
00B7-00BA	00BC-00BE	00C0-00D6	00D8-00F6	00F8-00FF
0100-167F	1681-180D	180F-1FFF		
200B-200D	202A-202E	203F-2040	2054	2060-206F
2070-218F	2460-24FF	2776-2793	2C00-2DFF	2E80-2FFF

3004-3007	3021-302F	3031-D7FF		
F900-FD3D	FD40-FDCF	FDF0-FE44	FE47-FFFD	
10000-1FFFD	20000-2FFFD	30000-3FFFD	40000-4FFFD	50000-5FFFD
60000-6FFFD	70000-7FFFD	80000-8FFFD	90000-9FFFD	A0000-AFFFD
B0000-BFFFD	C0000-CFFFD	D0000-DFFFD	E0000-EFFFD	

§ References

§ Informative References

[N4762]

Richard Smith. [Working Draft, Standard for Programming Language C+](https://wg21.link/n4762). 7 July 2018. URL: <https://wg21.link/n4762>